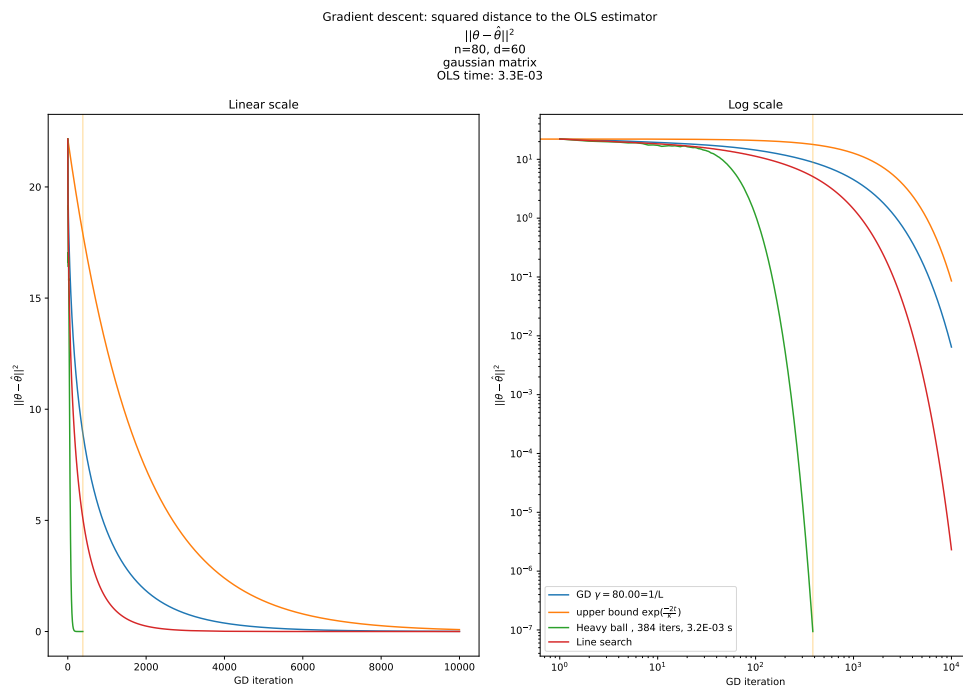


# FTML practical session 13

16 juin 2026



## TABLE DES MATIÈRES

|       |                                                 |   |
|-------|-------------------------------------------------|---|
| 1     | The Heavy-Ball method                           | 2 |
| 1.1   | Convergence rates of GD for convex functions    | 2 |
| 1.1.1 | Large condition numbers                         | 2 |
| 1.1.2 | Inertial methods                                | 2 |
| 1.2   | Simulation                                      | 2 |
| 1.3   | Analytical proof of $\gamma$ and $\beta$ values | 3 |
| 2     | Number of clusters for K-means                  | 4 |
| 2.1   | Visual method                                   | 4 |
| 2.2   | Algorithmic heuristics                          | 5 |
| 2.3   | Changing the data                               | 5 |

## 1 THE HEAVY-BALL METHOD

### 1.1 Convergence rates of GD for convex functions

We consider the optimization of a convex function  $f : \theta \rightarrow f(\theta)$  using a gradient descent (GD). In particular, we consider the **convergence speed** of GD. This speed can be expressed in several manners. For instance, as the distance between the iterate  $\theta_t$  and a minimizer  $\theta^*$  (of course assuming that this minimizer exists and is unique), as a function of the iteration number  $t$ . It is possible to show the following results for two-times differentiable convex functions :

- if  $H$  is invertible ( $\mu > 0$ ), we have a convergence rate in  $\exp(-\frac{2t}{\kappa})$ .
- if  $H$  is not invertible ( $\mu = 0$ ), we have a convergence rate in  $\mathcal{O}(\frac{1}{t})$  (probably one of the exercises of the project).

These rates are speed **upper bounds**, meaning that the convergence is at least as fast as those. Also note that these rates of convergence require the use of specific values of the learning rate  $\gamma$ , like for instance  $\frac{1}{L}$  (but other values might also be used, depending on the context). We also see that these rates depend on the **condition number** of the Hessian  $H$ . If we note  $\mu$  the smallest eigenvalue of the Hessian  $H$ , and  $L$  the largest, and if this Hessian is for instance symmetric and definite positive, then

$$\kappa = \frac{L}{\mu} \quad (1)$$

However, the condition number might be defined also for general matrices, and even functions.

[https://en.wikipedia.org/wiki/Condition\\_number](https://en.wikipedia.org/wiki/Condition_number)

#### 1.1.1 Large condition numbers

Hence, when  $\kappa$  is very large ( $\gg 1$ ), the convergence to the optimum might be very slow. Note that matrices with large condition numbers are not rare in large-scale machine learning and scientific computing applications. If the smallest eigenvalue  $\mu$  of a given matrix  $H$  is very small, or even 0 (which will happen as soon as  $H$  is not full rank),  $\kappa$  will be very large as soon as the largest eigenvalue  $L$  is not also very small.

#### 1.1.2 Inertial methods

When  $\kappa$  is large, some methods still exist in order to speed the convergence of gradient descent, such as **Heavy-ball**. This method consists in adding a **momentum term** to the gradient update term, such as the iteration now writes

$$\theta_{t+1} = \theta_t - \gamma \nabla_{\theta} f(\theta_t) + \beta(\theta_t - \theta_{t-1}) \quad (2)$$

where  $\beta$  and  $\gamma$  are real constants that should be tuned. The update  $\theta_{t+1} - \theta_t$  is then a combination of the gradient  $\nabla_{\theta} f(\theta_t)$  and of the previous update  $\theta_t - \theta_{t-1}$ . The goal of this method is to balance the effect of oscillations in the gradient. The heavy-ball method is called an **inertial method**. When  $f$  is a general convex function (not necessary quadratic), some generalizations exist, such as **Nesterov acceleration**. Many of the most famous variations of SGD, like RMSProp and Adam, optionally include such a momentum term.

### 1.2 Simulation

Using `exercise_1_heavy_ball/main.py`, try to manually find  $\gamma$  and  $\beta$  values such that the convergence is faster with the Heavy-Ball method than with a standard GD.

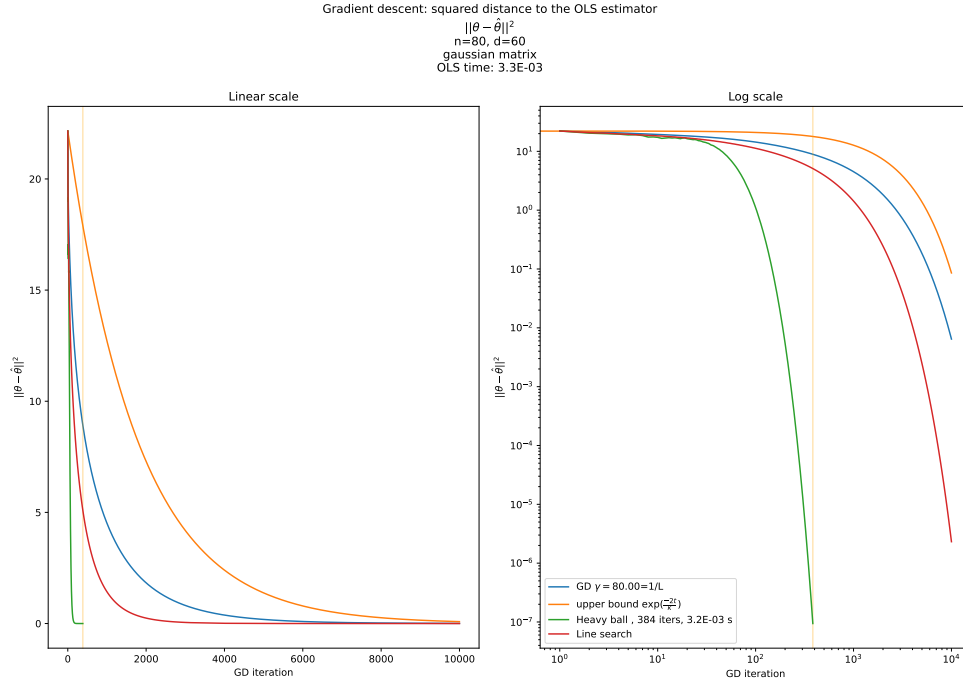


FIGURE 1 – Heavy ball vs GD, semilog scale

### 1.3 Analytical proof of $\gamma$ and $\beta$ values

Assuming  $\mu > 0$ , in a least squares problem, it is possible to show that the characteristic convergence time with the heavy-ball momentum term is  $\sqrt{\kappa}$  instead of  $\kappa$ , **if  $\beta$  and  $\gamma$  are tuned well**. With the heavy-ball momentum term, we can change the convergence (upper bound) from  $\mathcal{O}(\exp(-\frac{2t}{\kappa}))$  to  $\mathcal{O}(\exp(-\frac{2t}{\sqrt{\kappa}}))$ . If  $\kappa$  is large, which is the case we are interested in, this can be a significant improvement.

The update  $\theta_{t+1} - \theta_t$  is then a combination of the gradient  $\nabla_{\theta} f(\theta_t)$  and of the previous update  $\theta_t - \theta_{t-1}$ . This method might balance the effect of oscillations in the gradient. We will use these parameters :

$$\gamma = \frac{4}{(\sqrt{L} + \sqrt{\mu})^2} \quad (3)$$

and

$$\beta = \left( \frac{\sqrt{L} - \sqrt{\mu}}{\sqrt{L} + \sqrt{\mu}} \right)^2 \quad (4)$$

We keep the same notations as in the former practical sessions dedicated to the least squares problem.  $\mu$  is the smallest eigenvalue of the Hessian  $H$  and  $L$  is the largest. Assuming  $\mu > 0$  (strongly convex function), we will show that the characteristic convergence time with the heavy-ball momentum term is  $\sqrt{\kappa}$  instead of  $\kappa$ .

Let  $\lambda$  be an eigenvalue of  $H$  and  $u_{\lambda}$  a eigenvector for this eigenvalue. We are interested in the evolution of  $\langle \theta_t - \theta^*, u_{\lambda} \rangle$ .

We note

$$a_t = \langle \theta_t - \theta^*, u_{\lambda} \rangle \quad (5)$$

**Exercise 1 :** Show that

$$a_{t+1} = (1 - \gamma\lambda + \beta)a_t - \beta a_{t-1} \quad (6)$$

**Exercise 2:** Compute the constant-recursive sequence  $a_t$ , and show that there exists a constant  $C_\lambda$  that depends on the initial conditions, such that

$$\forall t, a_t \leq (\sqrt{\beta})^t C_\lambda \quad (7)$$

[https://en.wikipedia.org/wiki/Constant-recursive\\_sequence](https://en.wikipedia.org/wiki/Constant-recursive_sequence)

If  $u_i$  is a basis of orthonormal vectors with eigenvalues  $\lambda_i$ , we have that

$$\begin{aligned} \|\theta_t - \theta^*\|^2 &= \sum_{i=1}^d (\langle \theta_t - \theta^*, u_i \rangle)^2 \\ &\leq \sum_{i=1}^d (\sqrt{\beta})^{2t} C_{\lambda_i} \\ &= (\sqrt{\beta})^{2t} D \end{aligned} \quad (8)$$

with

$$D = \sum_{i=1}^d C_{\lambda_i} \quad (9)$$

We can now remark that

$$\begin{aligned} \sqrt{\beta} &= \frac{\sqrt{L} - \sqrt{\mu}}{\sqrt{L} + \sqrt{\mu}} \\ &= \frac{1 - \sqrt{\frac{\mu}{L}}}{1 + \sqrt{\frac{\mu}{L}}} \\ &\leq 1 - \sqrt{\frac{\mu}{L}} \\ &= 1 - \frac{1}{\sqrt{\kappa}} \end{aligned} \quad (10)$$

**Exercise 3:** Conclude

## 2 NUMBER OF CLUSTERS FOR K-MEANS

In this exercise we study heuristics to find relevant number of clusters for a k-means algorithm.

- folder : **exercice\_2\_k\_means/**
- template file : **k\_means\_heuristics.py**

A company has gathered data about its customers and would like to identify similar clients, in order to propose relevant products to new clients, based on their features. This can be represented as a clustering problem. The data are stored in **data.npy**. They are 4 dimensional. We would like to study methods to automatically find a relevant number of clusters in this dataset for the K-means algorithm.

### 2.1 Visual method

Is there a direct, visual way to have an idea of a relevant number of clusters for these 4-dimensional data ?

## 2.2 Algorithmic heuristics

We would like to see if algorithmic heuristics are consistent with the previous results.

Experiment with the following heuristics, metrics and tools in order to obtain a suggested number of clusters :

- possible metrics to monitor :
  - inertia :  
<https://scikit-learn.org/stable/modules/clustering.html#k-means>
  - silhouette score  
<https://scikit-learn.org/stable/modules/clustering.html#silhouette-coefficient>
  - Calinski-Harabasz score :  
<https://scikit-learn.org/stable/modules/clustering.html#calinski-harabasz-index>
- Knee detection tools :
  - <https://github.com/arvkevi/kneed>
  - <https://www.scikit-yb.org/en/latest/api/cluster/elbow.html>

## 2.3 Changing the data

You can generate the data again in order to study the behavior of the previous heuristics on a dataset with different statistical properties.